1-2

LLM을 사용하며 알아야 할 것들

학습 목차

- **LLM의 학습과 추론**
  - 사전 학습과 Instruction Tuning
  - 다음 토큰 예측과 생성 과정
- **Tokenization**
  - 토큰의 의미
  - 토큰 수와 비용
- **Context Window**
  - 컨텍스트 윈도우의 의미
  - 입력 품질과 출력 품질
- **LLM API의 대화 구조**
  - System, User, Assistant 역할
  - 모델 파라미터의 영향

© Scopelabs. All Rights Reserved

LLM의 학습과 추론

사전 학습과 Instruction Tuning (1)

LLM은 대규모 텍스트 데이터에서 언어의 패턴을 먼저 학습한 뒤, 사람의 지시에 답하는 방식으로 추가 조정된다.

- 사전 학습(Pre-training)은 웹 문서, 책, 코드, 대화 등 대규모 말뭉치에서 문장과 문장 사이의 통계적 패턴을 학습하는 단계이다.
- 이 단계의 기본 목표는 주어진 문맥 다음에 올 가능성이 높은 토큰을 예측하는 것이다.
- 사전 학습만 거친 모델은 지시를 따르기보다 그럴듯한 텍스트를 이어 쓰는 성향이 강하다.

```
입력: To be or not to be
출력: that is the question
```

사전 학습과 Instruction Tuning (2)

Instruction Tuning은 사용자의 요청을 이해하고 답변 형식으로 응답하도록 모델의 행동을 조정하는 과정이다.

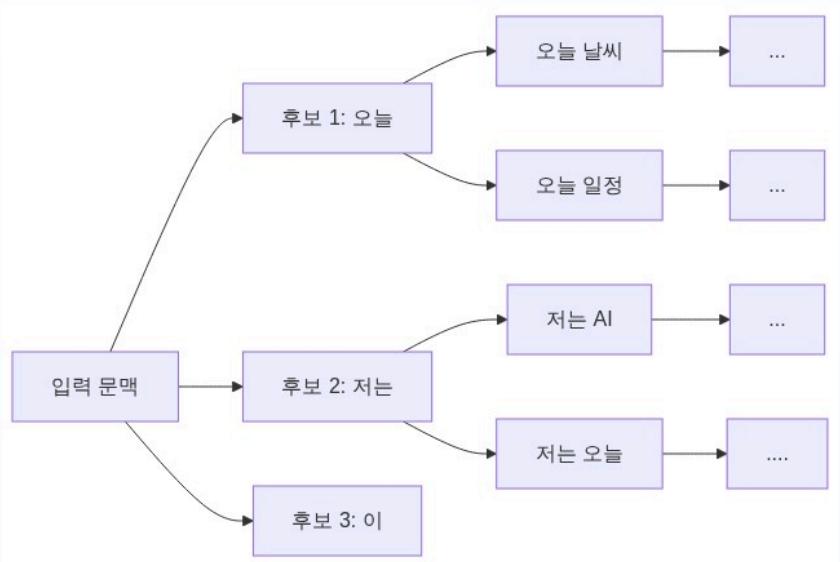
- 질문과 답변, 명령과 수행 결과, 대화 예시 등의 데이터로 모델을 추가 학습한다.
- 사용자의 입력을 단순한 문장 조각이 아니라 수행해야 할 요청으로 해석하도록 만든다.
- 우리가 API나 챗봇에서 사용하는 모델은 대체로 사전 학습 이후 지시 수행 능력을 강화한 모델이다.

```
{
  "instruction": "다음 문장을 한국어로 번역하세요.",
  "input": "The model follows user instructions.",
  "output": "모델은 사용자의 지시를 따릅니다."
}
{
  "instruction": "아래 메일을 세 줄로 요약하세요.",
  "input": "회의 일정, 참석자, 후속 작업이 포함된 긴 메일...",
  "output": "1. 회의는 금요일에 진행됩니다.\n2. 참석자는 AI TF입니다.\n3. 후속 작업은 자료 공유입니다."
}
```

다음 토큰 예측과 생성 과정 (1)

LLM의 추론은 한 번에 완성된 답을 꺼내는 방식이 아니라, 토큰을 하나씩 예측하며 문장을 만들어 가는 방식이다.

- 모델은 현재까지 입력된 문맥을 바탕으로 다음에 올 수 있는 토큰들의 확률 분포를 계산한다.
- 그중 하나의 토큰을 선택하고, 선택된 토큰을 다시 문맥 뒤에 붙인다.
- 이 과정을 반복하면 단어, 문장, 문단, 최종 답변이 순차적으로 생성된다.



다음 토큰 예측과 생성 과정 (2)

LLM API의 응답은 결국 이 반복 생성 과정을 서비스 형태로 감싼 결과이다.

1. 사용자가 메시지와 설정값을 API 요청으로 보낸다.
2. 서버의 모델이 입력 문맥을 읽고 다음 토큰 후보를 계산한다.
3. 선택된 토큰들이 누적되어 응답 메시지가 만들어진다.
4. 생성이 종료 조건에 도달하면 최종 답변이 API 응답으로 반환된다.

Tokenization

토큰의 의미

Tokenization은 인간의 말을 모델이 처리할 수 있는 작은 단위로 나누는 과정

- 나뉜 말의 단위를 토큰이라고 부르며, 의미를 가진 가장 작은 단위
- 토큰은 단어 하나와 항상 같지 않음.
  - 단어의 일부, 공백, 기호, 숫자 조각이 될 수 있음
  - 한국어는 조사와 어미, 띄어쓰기 특성 때문에 더 잘게 나뉠 수 있음

모델은 원문 문자열이 아니라 토큰에 의미가 부여된 벡터로 입력받음

- 토큰은 LLM이 문맥을 읽고 답변을 생성하는 기본 단위
  - 입력 프롬프트, 모델이 생성하는 답변도 토큰 단위로 계산

```
문장: LangChain으로 AI Agent를 만든다.

토큰 예시:
["Lang", "Chain", "으로", " AI", " Agent", "를", " 만든", "다", "."]
```

토큰 수와 비용 (1)

LLM API의 비용과 처리량은 보통 입력 토큰과 출력 토큰의 수에 의해 결정

- LLM이 대답을 하는데 사용되는 모든것이 입력 토큰 수에 포함
  - 입력 프롬프트, 시스템 프롬프트, 이전 대화 내용 등...
- 출력 토큰은 모델이 새로 생성한 답변의 길이에 해당
  - 생각하는 과정이 포함된 모델은 생각하는데 생성된 토큰도 포함
- 긴 프롬프트와 긴 답변은 더 많은 비용과 더 긴 응답 시간을 만든다.

토큰 수와 비용 (2)

토큰 갯수를 조절해 비용 아끼기

- 필요한 정보만 선별해서 입력한다.
- 반복되는 지시문과 대화 기록을 압축한다.
- 검색된 문서를 그대로 모두 넣기보다 관련성이 높은 일부만 넣는다.
- 답변 형식과 길이를 명확히 지정해 불필요한 출력을 줄인다.

Context Window

Context Window

Context Window - 모델이 한 번의 요청에서 읽고 고려할 수 있는 토큰 범위

Context Window

Context Window - 모델이 한 번의 요청에서 읽고 고려할 수 있는 토큰 범위

- 시스템 메시지, 사용자 메시지, 이전 대화, 도구 실행 결과가 모두 포함
- 모델은 현재 요청에 포함된 컨텍스트 밖의 정보를 직접 참조하지 못한다.

컨텍스트 윈도우는 LLM 애플리케이션의 설계 한계를 결정한다.

- 긴 문서를 다룰 때는 문서를 나누고 필요한 부분만 검색해서
- 장기 대화에서는 이전 대화를 요약하거나 중요한 상태만 별도로 저장
- Agent가 여러 도구를 사용할 때는 다음 판단에 필요한 정보만 유지해야 함

입력 품질과 출력 품질

LLM의 출력 품질은 모델 자체의 성능뿐 아니라 컨텍스트에 어떤 정보가 들어갔는지에 크게 좌우된다.

입력 품질과 출력 품질

LLM의 출력 품질은 모델 자체의 성능뿐 아니라 컨텍스트에 어떤 정보가 들어갔는지에 크게 좌우된다.

- 명확한 목표가 들어가면 답변의 방향이 안정된다.
- 충분한 배경 정보가 들어가면 추측과 환각이 줄어든다.
- 불필요하거나 충돌하는 정보가 많으면 모델의 판단이 흐려질 수 있다.

좋은 프롬프트는 모델에게 답을 맞히라고 요구하는 것이 아니라, 답을 만들 수 있는 조건을 제공한다.

- 역할과 목표를 분명히 제공한다.
- 필요한 배경 정보와 제약 조건을 함께 제공한다.
- 원하는 출력 형식과 평가 기준을 알려준다.
- 모호한 지시나 서로 충돌하는 요구를 줄인다.

LLM API의 대화 구조

System, User, Assistant 역할

Chat 기반 LLM API는 대화를 역할이 붙은 메시지 목록으로 표현

- System 메시지: 모델의 전반적인 역할, 규칙, 제약 조건을 정의
- User 메시지: 사용자의 질문, 요청, 입력 데이터를 전달
- Assistant 메시지: LLM이 생성한 데이터, 이후 대화에 추가되어 맥락 전달

역할 구분은 단순한 형식이 아니라 모델이 지시의 우선순위와 대화 흐름을 해석하는 기준

모델 파라미터의 영향

같은 입력에 대해 답변이 얼마나 다양하고 길고 예측 가능하게 생성될지를 조정

- Temperature: 다음 등장 가능한 토큰의 확률을 조정
- Top-p: 등장 가능한 토큰들의 누적 확률 제한을 조정
  - 낮은 Temperature, Top-p: 일관되고 보수적인 답변에 유리
  - 높은 Temperature, Top-p: 다양한 표현과 아이디어 생성에 유리
  - 단, 너무 높은 수치는 전혀 가능성 없는 토큰의 등장을 유발함으로 주의

설정	생성 경향	적합한 상황
낮은 temperature / 낮은 top-p	보수적이고 반복 가능성이 높음	사실 기반 답변, 코드 생성, 요약
낮은 temperature / 높은 top-p	핵심 방향은 안정적이나 표현 선택 폭은 확보	업무 문서, 설명문
높은 temperature / 낮은 top-p	제한된 후보 안에서 표현만 흔들림	가벼운 문장 변형
높은 temperature / 높은 top-p	다양하고 예측하기 어려움	아이디어 발산, 브레인스토밍